CAT-IA · Agente funcional EP2

Ingeniería de Soluciones con IA · ISY0101. Autoras: Antonella Cuvertino y Miriam Hammami. Docente: Sebastián Ignacio Sánchez Morales. Revisión: 21/10/2026.

Requerimiento y alcance

Nexo TI es una organización ficticia. El agente consulta procedimientos, continúa un caso, calcula prioridad y registra un borrador local pendiente de revisión. No envía mensajes ni modifica cuentas externas. El corpus mantiene siete documentos internos simulados y dos síntesis de guías de Mozilla; su origen y hashes están en data/sources.json.

La pauta EP2_ISY0101_Estudiante.pdf exige consulta, escritura, memoria corta y recuperación semántica de largo plazo, planificación explícita, al menos tres escenarios adaptativos, diagrama, decisiones y evidencia. La presentación es individual. El encargo representa 20% y la presentación 80% de EP2; la evaluación completa pondera 35% según la portada. La fecha límite se informa por AVA.

Retroalimentación y correcciones

El docente revisó el commit 32e3a7b y reportó pruebas del 19/10/2026: continuidad correcta en facturación; fallo de recuperación con «me cobraron dos veces»; contaminación de factura al preguntar por VPN; citas literales que no respaldaban la respuesta. Sus latencias de 11–17 segundos corresponden a su equipo. Son observaciones aportadas por el docente, no mediciones repetidas aquí.

Se añadió catia/guardrails.py. expand_query amplía patrones conocidos de cobro repetido; es expansión léxica, no recuperación semántica. select_context identifica señales conocidas, descarta antecedentes incompatibles y limita continuaciones al segmento reciente. No elimina el historial visual al cambiar de tema. validate_alignment revisa categoría, respuesta y cita respecto al tema único detectado; ante desacuerdo devuelve evidence_mismatch y no publica el borrador.

No se atribuye el fallo al tamaño del modelo. Falta un experimento entre modelos con iguales casos, recuperación y criterios para sostener esa explicación. Los controles implementados reducen problemas concretos, pero pueden omitir cambios sutiles y hechos falsos dentro del mismo tema.

Herramientas y escritura · IE1–IE2

catia/tools.py declara herramientas con langchain_core.tools.tool. Cada solicitud posee ToolContext aislado y los argumentos se vinculan a datos validados.

- tool_search_knowledge_base consulta el corpus respetando filtros y presupuesto. En modo híbrido también recupera entradas humanas anteriores del mismo hilo mediante embeddings. Devuelve identificadores y cantidad; no pensamientos privados.
- tool_check_sla_policy aplica prioridad y primera respuesta según impacto declarado. El modelo no puede modificar esos argumentos.
- tool_draft_response genera, comprueba citas y tema y escribe el borrador en SQLite case_notes mediante save_draft. Devuelve saved_draft_id y write_target. Es una herramienta de generación y

escritura efectiva, no solo un texto que simula haber guardado algo.

La herramienta de escritura requiere búsqueda y política completadas. Sin documentos no genera ni escribe un borrador. Si la validación falla no se registra como aceptado. La persistencia del historial corto ocurre al terminar la solicitud, incluso para registrar el resultado de una abstención. Todo borrador queda needs_review=true; guardarlo no es aprobarlo ni enviarlo.

Memoria corta · IE3

MemoryStore guarda sesiones UUID en SQLite. La ventana retiene tres intercambios; al excederla agrega extractos de hasta 240 caracteres de entradas humanas a un resumen limitado a 900. El resumen es determinista, no generado por un LLM. La ruta normal es runtime/memory.sqlite3, fuera de Git y ZIP.

El generador recibe como máximo dos entradas recientes recortadas, resumen filtrado y, si aplica, contexto semántico de entradas antiguas. Los borradores previos pueden verse en pantalla pero no se convierten en evidencia ni se insertan en la consulta de recuperación.

Un cambio de tema reconocido, como facturación a VPN, excluye antecedentes anteriores para esa consulta y muestra context_control.topic_change. Las consultas sin señal explícita conservan continuidad. Las mixtas o ambiguas requieren revisión: no hay detección universal de tema.

Memoria persistente y semántica · IE4

case_notes guarda como máximo veinte registros por hilo, independientes de la ventana corta. Cada registro contiene un UUID, la entrada humana redactada y el borrador pendiente. save_draft elimina registros anteriores al límite. DELETE /api/sessions/{id} borra tanto sesiones como case_notes del hilo.

Con CATIA_RETRIEVER=hybrid y nomic-embed-text disponible, la búsqueda combina TF-IDF y embeddings del corpus. search_notes obtiene vectores de la consulta y de las entradas humanas persistidas, filtra temas incompatibles y recupera hasta dos con coseno mínimo 0,65. Los identifica como semantic_history_ids en la observación. Sus textos se agregan a conversation_context.semantic_history como antecedentes no confiables, nunca como citas documentales.

La recuperación está aislada al mismo UUID y permite continuar tras reiniciar; no conecta hilos distintos ni implementa identidad multiusuario. Los vectores se calculan en memoria al consultar, no se persisten en una base vectorial. Para veinte registros por hilo es sencillo; con mayor volumen se necesitaría indexación y políticas de retención más completas.

En modo lexical no existe esta recuperación semántica de historial. Ese modo sirve como baseline o ensayo de bajo costo, pero no debe presentarse como evidencia del requisito semántico. reports/feedback_real.json compara recuperación original, expansión y modo híbrido y guarda la prueba de memoria con embeddings reales.

Planificación y decisiones · IE5–IE6

run_react implementa un ciclo explícito de tres herramientas distintas. Al inicio Ollama elige por JSON restringido entre búsqueda y política; cuando solo queda una acción válida, el código la ejecuta directamente. Borrador y escritura requieren ambas consultas completadas. La política se aplica también si falla el planificador. No hay acciones arbitrarias ni bucles abiertos.

run_crew crea tres Agent, tres Task y Process.sequential con contexto de tareas previas: Triage consulta SLA, Investigador busca y Redactor genera y registra. Cada rol tiene una sola herramienta. LocalStageLLM traduce una acción JSON validada al protocolo Action/Action Input de CrewAI. No se presenta como elección libre entre varias acciones en cada rol. Las observaciones finales se componen desde el resultado validado.

Escenarios reproducibles: continuidad del mismo tema; cambio de factura a VPN con descarte de contexto; ausencia de INT-FAC al filtrar externas; salida incorrecta inyectada que activa evidence_mismatch; proveedor desconectado simulado que activa provider_error. docs/13_prompts_demostracion.md indica pasos, salidas y explicación. Los fallos se detienen y no se disfrazan como éxito.

El respaldo a DemoGenerator requiere casilla explícita y un fallo de proveedor. Se identifica como fallback_demo, con requested_provider y mode distintos. Una salida desalineada no se convierte silenciosamente en simulación exitosa.

Arquitectura y flujo · IE7–IE9

docs/arquitectura.mmd y deliverables/arquitectura_EA2.pdf representan la implementación actual. El diagrama está también dentro de docs/presentacion_EA2_CAT-IA.pdf. El README contiene un diagrama Mermaid y enlaces a las instrucciones. La presentación se guarda en docs/ como pide la pauta, sin otra copia idéntica en deliverables/.

Flujo: interfaz → API y validación → selección de contexto → modo individual/equipo → consulta documental y de memoria → política → generación → citas y coherencia temática → registro local → revisión humana. Una entrada inválida devuelve 422. Ausencia de fuentes devuelve insufficient_context; desalineación devuelve evidence_mismatch; fallos de herramientas quedan en tool_events y se devuelve un resultado controlado.

ADR-01 · Framework y control

Decisión: herramientas tipadas LangChain y comparación opcional con CrewAI. Requerimiento: integrar consultas, escritura y decisiones visibles. Alternativa: funciones Python directas sin biblioteca; eran suficientes para EA1, pero no mostraban contratos de herramientas y roles de un framework. Costo: más dependencias y llamadas. No se exige un framework específico; la selección no garantiza escalabilidad empresarial.

ADR-02 · Memoria

Decisión: ventana, resumen extractivo y registros SQLite acotados por hilo. Requerimiento: continuidad, persistencia y borrado verificables. Alternativas: historial completo en el prompt o resumen por LLM; aumentan contexto, costo o riesgo de errores. Límite: truncamiento y aislamiento solo local, sin autenticación. Los locks serializan el hilo dentro de un proceso, no garantizan transacciones completas entre varios workers.

ADR-03 · Recuperación

Decisión: conservar TF-IDF como baseline y habilitar híbrido con embeddings para la demostración EP2. Requerimiento: recuperar paráfrasis y contexto persistente por similitud. Alternativa: solo reglas de sinónimos; mejora casos conocidos pero no cumple por sí sola recuperación semántica. Costo: descarga adicional, latencia y calibración. La similitud no es certeza; el filtro temático reduce algunos resultados ajenos sin resolver todos los falsos positivos.

ADR-04 · Validación

Decisión: combinar contrato JSON, pertenencia literal de citas y coherencia temática antes de escribir. Requerimiento: impedir el caso de una respuesta de factura con citas de VPN. Alternativa: juez LLM adicional por cada afirmación; demanda recursos y también puede equivocarse. Límite: las reglas no prueban implicación semántica y pueden rechazar paráfrasis válidas o aceptar hechos falsos del mismo tema. La revisión humana permanece obligatoria.

ADR-05 · Modelo local

Decisión: mantener qwen2.5:1.5b y ejecutar embeddings locales. Requerimiento: reproducibilidad en un equipo con recursos limitados y ausencia de credenciales cloud. Alternativa: un modelo mayor o remoto; requeriría medir calidad, costo y latencia antes de migrar. No se afirma que 1,5B cause los errores observados ni que un modelo mayor los elimine.

API, instalación y pruebas

README.md y .env.example describen variables y arranque. GET /api/health informa configuración; no prueba inferencia. POST /api/tickets/analyze admite session_id, orchestration, use_context y allow_demo_fallback. POST/GET/DELETE /api/sessions permiten crear, consultar y borrar memoria corta; el borrado también elimina registros persistentes.

El prompt de EA2 sigue siendo v4; los controles nuevos son código, no entrenamiento ni cambios ocultos al prompt. La traza informa el prompt utilizado. usage cuenta tokens de generación final, no todas las llamadas de planificación o embeddings.

tests/test_feedback.py reproduce fallos controlados, cambio de tema, citas ajenas, escritura y borrado. reports/tests_feedback.xml y coverage_feedback.json registran la suite actual. scripts.evaluate_feedback ejecuta Ollama y embeddings reales con P1–P3. reports/FEEDBACK_RESULTADOS.md separa resultados actuales y anteriores. Las pruebas con mocks no acreditan calidad del LLM.

Limitaciones y privacidad

Prototipo local con datos ficticios. Redacción parcial de patrones sensibles; no anonimización garantizada. Memoria y borradores persisten en runtime hasta borrado o límites; las descargas del navegador no se borran al eliminar un hilo. No hay autenticación, búsqueda entre usuarios ni conectores a sistemas reales. La fidelidad semántica completa y la generalización siguen pendientes de evaluación independiente.

Referencias

Pauta local: EP2_ISY0101_Estudiante.pdf, páginas 1–8, facilitada por la estudiante. Retroalimentación docente copiada en la conversación, revisión del commit 32e3a7b y pruebas declaradas del 19/10/2026.

LangChain Core 1.6.9: https://reference.langchain.com/python/langchain-core/tools/convert/tool. CrewAI 1.15.27: https://docs.crewai.com/en/concepts/tasks y https://docs.crewai.com/en/concepts/tools. Ollama: https://docs.ollama.com/api/embed. Modelos: https://ollama.com/library/qwen2.5 y https://ollama.com/library/nomic-embed-text. Dependencias exactas en requirements.txt y requirements-ea2.txt; la versión local de Ollama se consulta con ollama --version.

Guion individual · CAT-IA · EP2

Ingeniería de Soluciones con IA (ISY0101). Autoras: Antonella Cuvertino y Miriam Hammami. Docente: Sebastián Ignacio Sánchez Morales. Actualización: 21/10/2026.

La pauta indica presentación individual: cada integrante debe practicar el recorrido completo. El repositorio representa 20% y la presentación 80% de esta evaluación. La fecha límite se consulta en AVA.

Antes de presentar

Ejecuta instalar.bat e instalar_ea2.bat si es otro computador. Inicia Ollama con los modelos qwen2.5:1.5b y nomic-embed-text descargados. Configura CATIA_PROVIDER=ollama y CATIA_RETRIEVER=hybrid. En este computador portable, OLLAMA_URL usa 11435; una instalación estándar suele usar 11434. Ejecuta iniciar.bat y abre http://127.0.0.1:8000. Confirma el aviso LLM y recuperador híbrido. No basta que health responda para garantizar que el modelo genera.

Abre la presentación en docs/presentacion_EA2_CAT-IA.pdf y los archivos catia/orchestration.py, catia/tools.py, catia/guardrails.py y reports/FEEDBACK_RESULTADOS.md. Lee también docs/13_prompts_demostracion.md. Usa datos ficticios. Ensaya las consultas antes: la latencia varía por equipo; el profesor informó 11–17 segundos en el suyo, no es una promesa para el nuestro.

Diapositivas 1 y 2 · 40 segundos

«Soy [tu nombre]. Con mi compañera desarrollamos CAT-IA para la organización ficticia Nexo TI. El problema es atender solicitudes usando procedimientos dispersos sin perder los antecedentes del caso. El agente consulta documentos, determina la prioridad según una política y prepara un borrador que registra localmente. Una persona siempre lo revisa».

Diapositiva 3 · Diagrama · 45 segundos

«La solicitud entra por la API. El sistema selecciona el contexto del hilo y el modo de trabajo. Las herramientas consultan la política, recuperan documentos y generan un borrador. Antes de registrarlo, comprobamos citas y coherencia temática. Si falta evidencia o falla una herramienta, mostramos una abstención o un error controlado».

Señala las flechas, la memoria y las ramas de fallo. No enumeres módulos sin explicar qué información pasa entre ellos.

Diapositiva 4 · Decisiones de diseño · 40 segundos

«Elegimos LangChain para declarar herramientas con argumentos validados y CrewAI para mostrar roles con tareas secuenciales. SQLite resuelve la persistencia de este prototipo sin desplegar otro servidor. Ollama permite ejecutar localmente el modelo. Agregamos embeddings para comparar significado aproximado, aunque requieren un modelo adicional y evaluación en español».

Diapositiva 5 · Autonomía · 35 segundos

«El modelo puede elegir entre buscar o consultar la política al inicio del flujo individual, y redacta la respuesta. El código decide los pasos obligatorios, el número máximo de llamadas y los controles. En CrewAI los roles están predefinidos. No presentamos esta autonomía limitada como un agente que pueda ejecutar cualquier acción».

Diapositiva 6 · Memoria · 40 segundos

«La ventana conserva tres intercambios y un resumen extractivo. Además, registramos hasta veinte borradores pendientes con la entrada humana. En modo híbrido recuperamos por similitud entradas del mismo hilo, incluso si salieron de la ventana. Los borradores previos no se convierten en fuentes documentales. El borrado elimina ambos almacenamientos del hilo».

Diapositivas 7 y 8 · Retroalimentación · 50 segundos

«El profesor encontró una paráfrasis que TF-IDF no reconocía y una respuesta sobre facturación cuando el usuario había cambiado a VPN. Además, las citas eran auténticas pero no respaldaban esa respuesta. Incorporamos un control de cambio de tema y una validación temática adicional. La expansión por sinónimos y los embeddings son mecanismos distintos. No hemos demostrado que cambiar a un modelo más grande resuelva estos errores».

Diapositiva 9 · Demostración

Borra memoria, usa Agente individual y fuentes internas y externas. Ejecuta en orden:

1. «Mi factura tiene un cobro duplicado». Muestra el borrador, INT-FAC y saved_draft_id en las herramientas.
2. «¿Qué debo hacer en ese caso?» con antecedentes activados. Muestra el uso del hilo.
3. «no me conecta la VPN» sin borrar el hilo. Muestra el aviso de cambio de tema y verifica que no aconseje revisar facturas. Si se retiene el borrador, explica evidence_mismatch.
4. Cambia Fuentes consultadas a Solo externas. Envía «me cobraron dos veces» sin antecedentes. Explica por qué falta el procedimiento interno y el sistema se abstiene. Restaura el filtro.
5. Si queda tiempo, repite un caso claro con Equipo de 3 agentes y muestra Triage, Investigador y Redactor.

No ocultes una salida distinta de la esperada. Descarga el JSON y señala status, sources, tool_events, context_control, warning y timings. La pauta valora explicar un fallo con evidencia.

Diapositiva 10 · 30 segundos

«Las pruebas comprueban contratos y los escenarios reproducibles. Las inferencias guardadas muestran lo que ocurrió en esos casos, no una garantía general. El aporte es continuar el trabajo, hacer visibles las herramientas y detectar algunas respuestas desalineadas antes de que el operador las use».

Si te piden cambiar algo en vivo

Para top_k, usa el selector de fragmentos y explica que más contexto puede aumentar cobertura y ruido. Para impacto, cambia Personas afectadas y Servicio interrumpido: la prioridad sale de priority_for en catia/agent.py. Para ignorar antecedentes, desactiva la casilla y muestra context_control. Para ver el prompt, abre prompts/v4.txt: es una instrucción, no entrenamiento. Si editas código debes reiniciar la app; no prometas que se recarga automáticamente.

Preguntas para practicar

Consulta docs/14_preguntas_defensa.md. Practica cada respuesta con el archivo abierto y un ejemplo de tu aplicación. No memorices métricas históricas como si fueran resultados del código actual.

Guía de consultas, abstenciones y errores · CAT-IA

Actualización: 21/10/2026. Esta guía usa datos ficticios y no modifica cuentas reales. Los textos que escribes son consultas del usuario; prompts/v4.txt contiene las instrucciones del sistema.

Preparación común

Abre http://127.0.0.1:8000. Verifica el modo real o demo. Para probar recuperación semántica debe aparecer Híbrida y estar disponible nomic-embed-text. Selecciona Agente individual, Fuentes internas y externas y cuatro fragmentos. Borra el hilo antes de una prueba independiente. Desactiva el respaldo a demo cuando quieras observar un fallo real del proveedor.

El resultado esperado sirve para contrastar la ejecución; no debes afirmar éxito sin leerla. El modelo puede producir evidence_mismatch incluso si la recuperación fue correcta.

Caso A · Consulta conocida

Texto: «Mi factura tiene un cobro duplicado. ¿Qué información debo entregar para revisarlo?».

Espera información del procedimiento INT-FAC, sin reembolso garantizado. Muestra category, sources y las citas. La herramienta de borrador registra saved_draft_id en SQLite. El registro queda pendiente de revisión, no equivale a un ticket enviado.

Caso B · Continuidad

En el mismo hilo, activa antecedentes y pregunta: «¿Qué debo hacer en ese caso?».

Espera que utilice la consulta de factura anterior. Muestra memory.used y previous_turns. Sin antecedentes, la frase no identifica por sí sola el problema; puede abstenerse o recuperar un fragmento poco pertinente. Por eso no sirve como pregunta inicial precisa.

Caso C · Paráfrasis del profesor

Borra el hilo, desactiva antecedentes y escribe: «me cobraron dos veces».

Antes, TF-IDF no reconocía esa formulación. La versión revisada expande ese patrón a «factura cobro duplicado». El modo híbrido añade embeddings. Para distinguir sus aportes, mira retrieval_comparison en reports/feedback_real.json: compara consulta original, expansión y búsqueda híbrida sin expansión. No digas que una regla de sinónimos demuestra comprensión semántica general.

Prueba adicional: «me descontaron el doble». Las reglas cubren algunas formas, no todo el español. «Necesito revisar el importe repetido en el recibo» sirve para explorar una paráfrasis sin regla explícita; inspecciona la evidencia antes de darla por resuelta.

Caso D · Cambio de tema

Después del caso de factura, deja los antecedentes activados y escribe: «no me conecta la VPN».

Espera context_control.topic_change=true y memoria anterior ignorada para esta consulta. La respuesta debe tratar conectividad y apoyarse en INT-NET; no facturación. Si aparece evidence_mismatch, el sistema retuvo una salida que no pasó las comprobaciones. El aviso reduce contaminación conocida, pero no detecta todos los cambios de tema ni resuelve casos ambiguos.

Caso E · Generar una abstención controlada

Borra el hilo. Selecciona Solo externas. Desactiva antecedentes. Escribe: «me cobraron dos veces».

Resultado esperado: insufficient_context. Las fuentes externas del corpus son guías de Mozilla; no contienen el procedimiento de facturación de Nexo TI. El filtro excluye INT-FAC y el control temático descarta documentos ajenos. No es un fallo del servidor: la información necesaria no está disponible bajo ese filtro.

Corrección: vuelve a Internas y externas y repite. Explica que cambiar el filtro permite consultar la fuente adecuada; insistir con un prompt no crea un documento inexistente.

Caso F · Información que no existe

Texto: «¿Cuál es el estado real de mi reembolso de la factura 12345?».

CAT-IA no consulta la base real de pagos. Puede recuperar el procedimiento general de facturación, pero eso no demuestra el estado de una factura. Una respuesta que confirme un reembolso sería incorrecta aunque cite INT-FAC. La validación temática puede NO detectarla: el tema coincide, pero el hecho no está en la fuente. Este es un límite que debes explicar, no un caso con error garantizado.

Caso G · Entrada demasiado corta

Escribe «hola». El formulario exige al menos doce caracteres y no envía la consulta. Esto es validación de entrada, no una alucinación ni un error de recuperación.

Para observar el contrato HTTP, en /docs envía POST /api/tickets/analyze con {"text":"hola"}. La respuesta esperada es 422. No hagas esta prueba con datos personales.

Caso H · Falla de proveedor reproducible

No existe un texto que garantice que Ollama falle. Un error de conexión se simula como condición del entorno, no pidiendo «genera un error» al chat.

Ejecuta: .\.venv\Scripts\python.exe -m pytest -q tests/test_feedback.py -k insufficient_source_and_provider_failure

La prueba inyecta una conexión fallida sin apagar tu Ollama ni cambiar .env. Comprueba provider_error y que no haya fallback silencioso. Para el caso de citas auténticas pero respuesta equivocada ejecuta: .\.venv\Scripts\python.exe -m pytest -q tests/test_feedback.py -k literal_quote

Es un ensayo controlado con una salida defectuosa, no una inferencia real fallida. El borrador se retiene con evidence_mismatch.

Cómo explicar por qué no recibiste la información

- insufficient_context: la recuperación no entregó evidencia suficiente; revisa consulta, fuentes, filtro y umbral.
- evidence_mismatch: había citas reales, pero categoría, respuesta o citas no coinciden con el tema reconocido. No se ofrece el borrador sospechoso.
- provider_error: falló una dependencia o llamada; revisa el proveedor y los eventos. El mensaje general no demuestra que la causa sea falta de RAM.
- invalid_output: la salida incumplió un contrato u otra validación. Examina qué herramienta falló.
- 422: los datos de entrada no cumplen el contrato; corrige campos o longitud.
- draft: existe un borrador que pasó los controles programados. No significa certeza ni aprobación de fidelidad.

Qué mostrar en el JSON descargado

Consulta status, warning, context_control, memory.used, sources, tool_events, requested_provider, mode y fallback_used. source.score es similitud de recuperación: no lo presentes como porcentaje de certeza. La prueba de calidad consiste en comparar las afirmaciones con los fragmentos y el requerimiento, además de pasar los controles automáticos.

Preguntas y respuestas para la defensa individual

CAT-IA · ISY0101 · Ingeniería de Soluciones con IA · 21/10/2026. Ambas integrantes deben poder explicar todo el proyecto. Las respuestas describen la implementación, no una autonomía ilimitada.

1. ¿Qué problema organizacional resuelven?

Ayudamos al operador de la empresa ficticia Nexo TI a consultar procedimientos, continuar un caso y preparar una respuesta con evidencia. No medimos una reducción real de tiempos de una empresa; el beneficio es propuesto y el comportamiento se prueba con casos simulados.

2. ¿Qué es un LLM y cuál usan?

Un modelo de lenguaje genera texto a partir del contexto. Usamos qwen2.5:1.5b mediante Ollama local. No lo entrenamos ni modificamos sus pesos. La configuración del modelo está en .env y la llamada en catia/generation.py.

3. ¿Qué diferencia hay entre prompt, memoria y entrenamiento?

El prompt da instrucciones para una ejecución. La memoria aporta antecedentes recuperables. Entrenar modifica parámetros del modelo. Cambiar el prompt o recordar un ticket no significa entrenar nuestra IA.

4. ¿Qué es RAG?

Recuperar fragmentos relevantes y entregarlos al modelo para generar una respuesta respaldada. En CAT-IA el flujo pasa por Retriever.search, la herramienta de búsqueda y el generador. Si faltan fuentes, no debemos completar la respuesta con hechos inventados.

5. ¿Cuáles son sus fuentes internas y externas?

Siete documentos internos simulados de Nexo TI, como facturación y conectividad; dos síntesis de guías de Mozilla. data/sources.json registra origen, versión y SHA-256. No hay conexión a cuentas de Microsoft o Gmail ni navegación web en cada consulta.

6. ¿Por qué falló «me cobraron dos veces»?

TF-IDF compara términos y sus pesos; esa paráfrasis no compartía suficiente vocabulario con el documento. Se agregó expansión explícita y se compara con recuperación híbrida usando embeddings. Son soluciones distintas y el informe registra sus resultados.

7. ¿Qué es un embedding?

Un vector numérico producido por un modelo para representar texto. Comparamos vectores normalizados mediante producto escalar, equivalente al coseno. La similitud orienta la búsqueda, pero no es probabilidad de verdad. Usamos nomic-embed-text; su utilidad en español debe medirse con nuestros casos.

8. ¿Cómo funcionan las memorias?

La corta guarda tres intercambios y un resumen extractivo. La persistente conserva hasta veinte registros por hilo en case_notes. En modo híbrido se recuperan hasta dos entradas humanas con similitud de al menos 0,65. Se usa el mismo UUID y se conserva al reiniciar; no se buscan datos de otro hilo. SQLite guarda datos; no calcula por sí mismo los embeddings.

9. ¿Por qué no usan respuestas anteriores como evidencia?

Un borrador anterior puede contener errores. Guardarlo no lo convierte en un hecho. La búsqueda de memoria devuelve entradas humanas como antecedentes no confiables; solo los documentos recuperados pueden citarse.

10. ¿Qué cambió ante la consulta de VPN después de factura?

select_context detecta señales temáticas conocidas, ignora antecedentes incompatibles y lo muestra en pantalla. Las continuaciones posteriores se restringen al segmento reciente. Las consultas mixtas o sin palabras reconocidas pueden escapar al control; borrar el hilo sigue siendo útil.

11. ¿El modelo pequeño causó el error?

No lo demostramos. Para afirmarlo necesitaríamos comparar modelos bajo los mismos documentos, consultas, configuración y criterios de calidad. La retroalimentación evidencia contaminación de contexto y una validación insuficiente; no identifica por sí sola una causa asociada al tamaño.

12. ¿Dónde está la herramienta de escritura?

tool_draft_response en catia/tools.py genera, valida y registra el borrador mediante MemoryStore.save_draft. Escribe en case_notes y devuelve saved_draft_id. No envía un correo ni modifica sistemas externos. Si la validación falla, no registra un borrador aprobado.

13. ¿Qué decide el modelo y qué decide el código?

En ReAct el modelo elige el orden inicial de búsqueda o SLA y redacta. El código fija herramientas permitidas, argumentos, precondiciones, tres llamadas, política, contexto y validación. CrewAI organiza roles en secuencia, cada uno con una herramienta permitida. La persona aprueba la respuesta final.

14. ¿Qué es ReAct aquí?

Un ciclo acotado de observar, seleccionar una acción, ejecutar una herramienta y usar su resultado. No exponemos pensamientos internos; mostramos acciones y observaciones. Es una implementación limitada sobre herramientas LangChain, no un planificador abierto.

15. ¿Por qué LangChain y CrewAI?

LangChain aporta contratos de herramientas con validación. CrewAI ofrece Agent, Task y Process.sequential para comparar un equipo de roles. No necesitamos dos frameworks para todo problema; aquí justificamos la comparación académica. CrewAI añade dependencias y llamadas; medir latencia permite discutir su costo.

16. ¿Cómo evitan bucles y acciones peligrosas?

Existe una lista permitida, no se repite una herramienta y el borrador requiere búsqueda y política previas. ReAct limita tres herramientas; CrewAI limita iteraciones y llamadas. No hay herramienta de ejecución arbitraria ni envío a clientes.

17. ¿Una cita real garantiza fidelidad?

No. validate_evidence comprueba que exista el fragmento y que la cita sea literal. validate_alignment añade consistencia temática. Ninguna prueba todas las afirmaciones ni detecta necesariamente una promesa falsa de reembolso. La fidelidad requiere revisar si lo dicho se desprende realmente de la fuente.

18. ¿Qué significan precisión y recall de recuperación?

Precisión: proporción de documentos recuperados que son relevantes. Recall: proporción de documentos relevantes esperados que fueron recuperados. Requieren referencias etiquetadas. No son lo mismo que exactitud de categoría ni que fidelidad del texto generado.

19. ¿Qué ocurre si falla una herramienta?

Se registra un evento de error y se devuelve una salida controlada. Sin contexto se abstiene. Ante desalineación devuelve evidence_mismatch. El respaldo a demo solo se activa si el operador lo permite y se etiqueta como simulación; no se cuenta como éxito de Ollama.

20. ¿Qué pruebas muestran decisiones adaptativas?

Continuidad de facturación, cambio a VPN con descarte de contexto, filtro que excluye INT-FAC y falla de proveedor inyectada. tests/test_feedback.py comprueba esos comportamientos. reports/feedback_real.json registra inferencias reales separadas de los tests simulados.

21. ¿Por qué SQLite y no una base vectorial externa?

El volumen es pequeño y SQLite simplifica la instalación. Los embeddings se calculan con Ollama y se comparan en memoria; los registros persisten en SQLite. Un volumen grande justificaría índices vectoriales, control de acceso, migraciones y retención más robusta. No afirmamos escalabilidad empresarial actual.

22. ¿Qué muestra la traza?

Herramienta, actor, resultado resumido, estado y duración, además de identificadores. No es razonamiento privado ni certeza. El JSON descargado permite relacionar el resultado con fuentes, controles y proveedor.

23. ¿Qué implica un resultado draft?

Que pasó los controles implementados y está pendiente de revisión. No equivale a respuesta verdadera, enviada o aprobada. Un error factual dentro del mismo tema puede pasar los controles heurísticos.

24. ¿Cómo responderías una tarea nueva del docente?

Primero aclaro el requerimiento y el modo. Ejecuto, inspecciono fuentes y herramientas, comparo respuesta con evidencia y explico la rama seguida. Si falla, identifico la etapa por status y eventos y propongo una corrección concreta. No invento una causa basándome solo en el texto final.

Revisión docente · Evidencia actual

Actualización documental: 21/10/2026.

Software

51 pruebas; 0 fallos; 0 omitidas. Cobertura de líneas: 88.8 %. Fuentes: tests_feedback.xml y coverage_feedback.json.

Cobertura: ejecución separada sin la prueba CrewAI. La suite completa sí la incluye; se separó para evitar el costo de instrumentar su catálogo de importación.

Los ensayos de fallo de proveedor y salida desalineada inyectan errores controlados. No son inferencias fallidas reales. Las pruebas de CrewAI ejecutan su framework con generador simulado.

Regresión con Ollama y embeddings reales

Fecha de ejecución: 2026-10-21T20:29:11.672310+00:00. Modelo: qwen2.5:1.5b; embeddings: nomic-embed-text.

- P1a: draft; categoría Facturación; cambio de tema False; tiempo 60.0 s.
- P1b: draft; categoría Facturación; cambio de tema False; tiempo 65.4 s.

• P2: draft; categoría Facturación; cambio de tema False; tiempo 66.1 s.

• P3: evidence_mismatch; categoría No determinado; cambio de tema True; tiempo 67.9 s.

Regresión completa: False. Registros recuperados por similitud: 1.

Comparación de recuperación

me cobraron dos veces: lexical_original: sin fragmentos; lexical_expanded: INT-FAC:0; hybrid: INT-SLA:90, INT-COM:0, INT-NET:0, INT-SLA:0.

me descontaron el doble: lexical_original: sin fragmentos; lexical_expanded: INT-FAC:0; hybrid: INT-SLA:90, EXT-COOKIE:0, INT-SUG:0, INT-COM:0.

no me conecta la VPN: lexical_original: INT-NET:0; lexical_expanded: INT-NET:0; hybrid: INT-NET:0, INT-SEC:0, INT-ACC:0, EXT-NET:0.

necesito revisar el importe repetido en el recibo: lexical_original: INT-FAC:0, EXT-NET:0, EXT-COOKIE:0; lexical_expanded: INT-FAC:0, EXT-NET:0, EXT-COOKIE:0; hybrid: INT-FAC:0, EXT-NET:0, EXT-COOKIE:0, INT-SLA:90.

Interfaz con simulador

Resultado: True. Cambio de factura a VPN: aviso visible y antecedentes excluidos; Solo externas + cobro: abstención visible y sin fuentes ajenas; Vista móvil sin desbordamiento; Borrado de sesión desde la interfaz. Fuente: browser_feedback.json.

Interpretación

La expansión es una regla léxica; los embeddings producen recuperación semántica aproximada. Las coincidencias de los casos conocidos no demuestran generalización. El control temático no verifica todas las afirmaciones.

En la ejecución registrada, P3 devolvió evidence_mismatch: el sistema retuvo el borrador, no resolvió satisfactoriamente la consulta de VPN. La regresión completa figura False por ese motivo. La recuperación híbrida sin expansión no ubicó INT-FAC entre los primeros cuatro para algunas paráfrasis; no se afirma superioridad general del modelo de embeddings.

El profesor informó 11–17 segundos en su equipo durante pruebas del 19/10/2026. Esa cifra no se compara causalmente con otra máquina ni demuestra un efecto del tamaño del modelo. No se realizó comparación entre tamaños de LLM.

Los informes EA2_RESULTADOS.md, tests_ea2.xml y ea2_ollama_*.json describen la versión anterior al control temático. No deben atribuirse a esta revisión.